

Farhan Akbor Khan

Assignment - 04

Module - 04

Introduction to Programming Language with AI (JavaScript + Node.js)

Q1. JavaScript Coding Practice

Answer:

i) Check Whether a Number is Even or Odd

Problem Statement: Write a JavaScript program that takes a number and checks whether the number is even or odd.

Code:

```
let number = 15;

if (number % 2 === 0) {
    console.log(number + " is Even");
} else {
    console.log(number + " is Odd");
}
```

Screenshot:

```
3  let number = 15;
4
5
6  if (number % 2 === 0) {
7      console.log(number + " is Even");
8  } else {
9      console.log(number + " is Odd");
10 }
11
```

Problems Output Debug Console Terminal Ports powershell

PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> node "Q1.js"
15 is Odd
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04>

Explanation: The `%` operator gives the remainder after division. If a number is divisible by 2, its remainder is 0, so the number is even. Otherwise, it is odd.

ii). Find the Largest of Three Numbers

Problem Statement: Write a JavaScript program that takes three numbers and finds the largest number among them.

Code:

```
let a = 25;
let b = 40;
let c = 30;
let largest;
if (a >= b && a >= c) {
    largest = a;
}
else if (b >= a && b >= c) {
    largest = b;
}
else {
    largest = c;
}
console.log("The largest number is: " + largest);
```

Screenshot:

```
3
4  let a = 25;
5  let b = 40;
6  let c = 30;
7  let largest;
8  if (a >= b && a >= c) {
9      largest = a;
10 }
11 else if (b >= a && b >= c) {
12     largest = b;
13 }
14 else {
15     largest = c;
16 }
17
18 console.log("The largest number is: " + largest);
```

Problems Output Debug Console Terminal Ports powershell ⚠

```
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> node "Q2.js"
The largest number is: 40
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> 
```

Explanation: The program compares the three numbers using **if-else** conditions. The number that is greater than or equal to the other two numbers is stored in the **largest** variable.

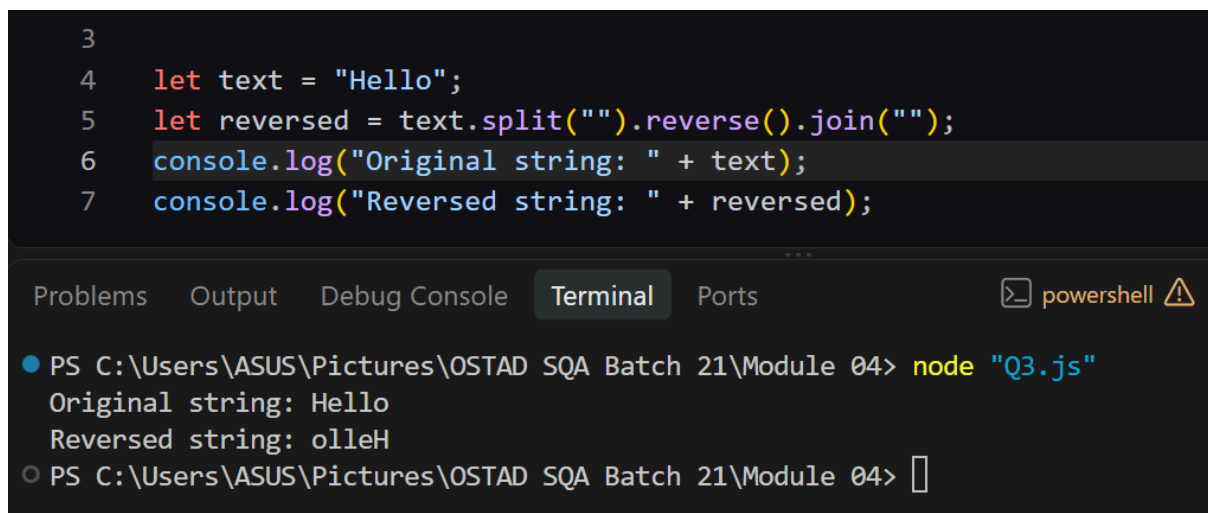
iii) Reverse a String

Problem Statement: Write a JavaScript program that takes a string and prints the string in reverse order.

Code:

```
let text = "Hello";
let reversed = text.split("").reverse().join("");
console.log("Original string: " + text);
console.log("Reversed string: " + reversed);
```

Screenshot:



```
3
4  let text = "Hello";
5  let reversed = text.split("").reverse().join("");
6  console.log("Original string: " + text);
7  console.log("Reversed string: " + reversed);
```

Problems Output Debug Console Terminal Ports powershell ⚠

```
● PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> node "Q3.js"
  Original string: Hello
  Reversed string: olleH
○ PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> 
```

Explanation: First, `split("")` converts the string into an array of characters. Then `reverse()` reverses the array. Finally, `join("")` combines the characters back into a string.

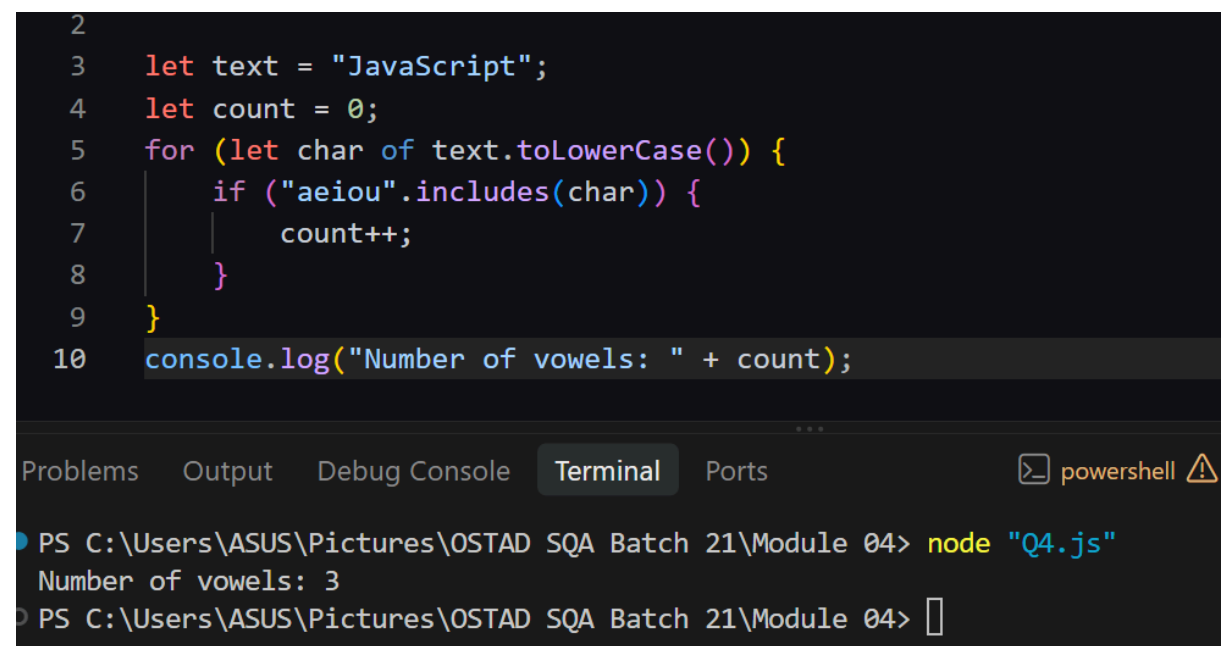
iv) Count Vowels in a String

Problem Statement: Write a JavaScript program that counts the number of vowels in a given string.

Code:

```
let text = "JavaScript";
let count = 0;
for (let char of text.toLowerCase()) {
    if ("aeiou".includes(char)) {
        count++;
    }
}
console.log("Number of vowels: " + count);
```

Screenshot:



```
2
3  let text = "JavaScript";
4  let count = 0;
5  for (let char of text.toLowerCase()) {
6      if ("aeiou".includes(char)) {
7          count++;
8      }
9  }
10 console.log("Number of vowels: " + count);
```

Problems Output Debug Console Terminal Ports powershell ⚠

```
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> node "Q4.js"
Number of vowels: 3
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> 
```

Explanation: The program checks each character of the string using a `for...of` loop. If the character is one of `a, e, i, o, u`, then the count variable is increased by one.

For `"JavaScript"`, the vowels are: `a, a, i`

So the total number of vowels is `3`.

v) Remove Duplicate Values from an Array

Problem Statement: Write a JavaScript program that removes duplicate values from an array and prints the resulting array.

Code:

Traditional way:

```
let numbers = [1, 2, 2, 3, 4, 4, 5];

let unique = [];

for (let i of numbers){

    if (!unique.includes(i)){

        unique.push(i);

    }

}

console.log(unique);
```

Easy way:

```
let numbers = [1, 2, 2, 3, 4, 4, 5];
let uniqueNumbers = [...new Set(numbers)];
console.log("Original array:", numbers);
console.log("Array without duplicates:", uniqueNumbers);
```

Screenshot:

```
2
3
4 let numbers = [1, 2, 2, 3, 4, 4, 5];
5 let uniqueNumbers = [...new Set(numbers)];
6 console.log("Original array:", numbers);
7 console.log("Array without duplicates:", uniqueNumbers);
```

Problems Output Debug Console Terminal Ports powershell

```
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> node "Q5.js"
Original array: [
  1, 2, 2, 3,
  4, 4, 5
]
Array without duplicates: [ 1, 2, 3, 4, 5 ]
PS C:\Users\ASUS\Pictures\OSTAD SQA Batch 21\Module 04> 
```

Explanation: A `Set` stores only unique values, so duplicate values are automatically removed. The spread operator `...` converts the `Set` back into an array.

Therefore:

[1, 2, 2, 3, 4, 4, 5]

becomes:

[1, 2, 3, 4, 5]

Q2. Introduction to Node.js

Answer:

i) What is Node.js?

Node.js is a JavaScript runtime that allows us to run JavaScript outside the browser.

Normally, JavaScript runs inside browsers like Chrome. With Node.js, we can run JavaScript directly on our computer/server.

For example, QA engineers can use Node.js to run automation scripts using tools like Playwright or Cypress.

ii) How does Node.js work?

Node.js uses the V8 JavaScript engine (the same engine used by Chrome) to execute JavaScript.

It is also event-driven and non-blocking, which means it can handle multiple tasks without waiting for one task to completely finish before starting another.

iii) Advantages of Node.js for QA Automation

Some important advantages are:

- Fast execution because it uses the V8 engine.
- Easy to write automation scripts using JavaScript/TypeScript.
- Large ecosystem of testing tools such as Playwright, Cypress, and WebdriverIO.
- Async/non-blocking nature is useful when tests involve waiting for pages, APIs or files.
- Easy integration with CI/CD pipelines like GitHub Actions and Jenkins.

iv) Browser JavaScript vs Node.js:

Browser JavaScript

Runs inside a browser

Mainly works with webpages/DOM

Has window, document

Cannot directly access local files

Used for frontend interaction

Node.js

Runs outside the browser

Can work with files, servers,
OS etc.

Has process, fs, path etc.

Can access files through Node
APIs

Used for backend, scripts,
automation etc.

In short:

Browser JavaScript is mainly used to interact with web pages, while Node.js allows JavaScript to run outside the browser and interact with the system.